

Beyond Generation: Evaluating Grounding and Hallucination in a Study Skills RAG Assistant

Ong, John Laurence E.
College of Computer Studies and
Engineering
Jose Rizal University
Mandaluyong City, Philippines
johnlaurence.ong@my.jru.edu

Abstract—The reliability of generative artificial intelligence is important in educational applications where unsupported information may affect students’ understanding. This study presents a Retrieval-Augmented Generation (RAG) assistant designed for the Study Skills domain under SDG 4: Quality Education. The system processes domain-specific material through document loading, 500-character text chunking with 50-character overlap, all-MiniLM-L6-v2 embeddings, ChromaDB vector storage, and top-three semantic retrieval before generating responses using openai/gpt-oss-20b through Groq. The system was evaluated using in-domain and out-of-domain queries under different temperature and grounding configurations. Results showed that the grounded baseline generated more concise and controlled responses, while the ungrounded configuration produced more elaborate outputs. No hallucinated response was observed for the tested out-of-domain query, as both configurations recognized insufficient contextual information.

Keywords—Retrieval-Augmented Generation, RAG, Study Skills, Grounded Response Generation, Hallucination Analysis

I. INTRODUCTION

Many students still struggle to develop effective learning habits, organize their study time, retain information, and apply appropriate strategies when preparing for academic tasks. Recent evidence indicates that this remains an important educational concern. The Organisation for Economic Co-operation and Development (OECD) reported an unprecedented decline in student performance in PISA 2022, with average scores across OECD countries decreasing by approximately 15 points in mathematics and 10 points in reading compared with 2018 [1]. In addition, recent research has shown that study-related skills such as time management are positively associated with academic achievement [2], while self-regulated learning practices can improve students’ learning performance and goal achievement [3].

These concerns support the selection of Study Skills under Sustainable Development Goal 4 (SDG 4): Quality Education as the domain of this Retrieval-Augmented Generation (RAG) activity. The domain was selected because effective study skills can help students improve how they organize, understand, remember, and review academic information. The source material used in this implementation focuses on learning environments, critical

reading, memory techniques, group study, note-taking, and effective study habits, which are directly related to improving students’ learning practices.

To construct the knowledge base of the RAG system, one PDF file is uploaded into my_data/ folder with the filename Chapter-5-Study-Skills. This document serves as the domain-specific knowledge source that enables the RAG chatbot to retrieve relevant information and provide responses about study strategies based on the uploaded material.

II. SYSTEM ARCHITECTURE

This activity presents the implementation of Retrieval-Augmented Generation (RAG) pipeline to process and retrieve information from the Study Skills knowledge base and generate context-grounded responses to user queries.

A. Document Loading Stage

The pipeline begins with the document loading stage, where the Study Skills PDF file is stored inside the folder of my_data/ directory. This PDF file is loaded using the DirectoryLoader from LangChain Community. In this stage, it converts uploaded PDF files into document objects that can be processed by the other components of the system.

B. Text Chunking Stage

After loading the document, it will proceed to the text chunking stage using RecursiveCharacterTextSplitter. The text is then divided into smaller chunks with a chunk_size of 500 characters and a chunk_overlap of 50 characters. The purpose of this is to make the documents earlier to process and retrieve. The overlap also helps to preserve the information that may continue from one chunk to the next, reducing the possibility of losing the context at the boundaries of the chunk.

C. Embedding

After chunking, the next stage is the embedding generation process. Wherein each document chunk is passed to the Hugging Face all-MiniLM-L6-v2 embedding model. This model converts the text chunks into a numerical or what we called vectors which represents the semantic meaning of the word. Instead of comparing documents through exact works, the embedding allows the system to identify chunks that are semantically related to the meaning of the user’s query.

D. ChromaDB

After the embeddings are generated, they are stored in ChromaDB, which serves as the vector database of the system. ChromaDB stores the embeddings together with their corresponding document text and metadata. This creates a collection of information from the PDF files that can be searched and retrieved relevant information based on the user's query. The vector store is configured as a retriever with the $k=3$, which means that the system retrieves the three most relevant document chunks for every user query.

E. Query Processing

When a user submits a question, the query processing stage starts. The user query is also converted into an embedding using the same all-MiniLM-L6-v2 model. This query vector is then compared with the stored document vectors in the ChromaDB through semantic similarity. The retriever then identifies and returns the top three chunks that are most likely related to the meaning of the question.

F. Prompt Construction

After getting the retrieved chunks, these are then passed to the prompt construction stage. The three retrieved chunks are then inserted to the {context} placeholder of the system prompt, while the user's question is inserted into the {input} placeholder. The prompt also contains instructions that require the language model to answer only using the retrieval context. If the information needed to answer the question is not available in the context, the model is instructed to respond that it cannot answer based on the provided domain data. These instructions can be configured on your own.

G. Language Generation

The complete prompts are then forwarded to the language generation through ChatGroq. The Groq hosted language model reads both the user query and retrieved chunks and generates the final response. In this activity, the generation uses the openai/gpt-oss-20b.

H. Output

This stage returns the generated answer to the user. The system can also display the source metadata of each retrieved chunk, allowing the user to see which pdf files contain the supporting context for the generated response.

III. IMPLEMENTATION DETAILS

This implementation uses parameters that define how the RAG system processes the Study Skills PDF file. The main parameters that are configured in the system are the chunk size, chunk overlap, top-k retrieval value, and temperature. These values control the amount of text contained in each chunk, the preservation of context between chunks, the number of relevant chunks retrieved from ChromaDB, and the variability of the language model's generated responses

TABLE I. PARAMETERS DETAILS

Parameters Implemented		
Parameter	Value	Purpose
Chunk Size	500	Defines the approximate

		maximum size of each text chunk produced from the loaded documents.
Chunk Overlap	50	Repeats a small portion of text between neighboring chunks to help preserve context across chunk boundaries.
Top-K Retrieval	3	Instructs the retriever to return the three most relevant document chunks from ChromaDB for each user query.
Temperature	0 and 1	Controls how the LLM generates responses. Higher values make the answers more varied.

In the document-processing stage, the 500-character chunk size divides the extracted document text into smaller and more manageable sections, while the 50-character overlap helps maintain contextual continuity between adjacent chunks. During retrieval, the top-k value of 3 limits the context to the three chunks considered most semantically relevant to the user's query. Finally, the temperature value of 0 and 1 is applied during response generation to control and observe the changes variability of the output produced by the Groq-hosted language model.

TABLE II. OTHER CONFIGURATION IMPLEMENTED

Other Configuration		
		Purpose
Language Model	openai/gpt-oss-20b	Generates the final response using retrieval document context and user query
Embedding Model	all-MiniLM-L6-v2	Converts document chunks and user queries into numerical vector representations for semantic comparison. .
Vector Store	ChromaDB	Stores document embeddings, text chunks, and their associated metadata for retrieval.
RAG Framework	LangChain 0.3.27	Connects the document processing, retrieval, prompting, and response-generation components.

IV. EXPERIMENTAL RESULTS HALLUCINATION ANALYSIS

The RAG system was evaluated using in-domain and out-of-domain queries to examine the effect of temperature, grounding, and fallback instructions on the generated responses. Two primary configurations were compared: the baseline configuration, which used a temperature of 0 with grounding and fallback rules, and the ungrounded configuration, which used a temperature of 1.0 without grounding and fallback rules. The same queries were used across the configurations to observe differences in response detail and the model's behavior when the required information was not available in the provided Study Skills context.

For the in-domain evaluation, the query "What is memory?" was submitted to both configurations. At a temperature of 0 with grounding and fallback rules, the model generated the response, *"Memory is the ability to remember past experiences, and a record of the learning process."* The response was concise and directly addressed the query. In comparison, at a temperature of 1.0 without grounding and fallback rules, the model provided a more detailed response that discussed memory as a record of the learning process and additionally introduced concepts such as neuroplasticity, neural pathways, and long-term memory encoding. This demonstrates that the ungrounded configuration produced a more elaborate response. However, the additional statements cannot be considered grounded unless they are supported by the retrieved document chunks.

TABLE III. IN- DOMAIN QUERY RESULTS

Temperate	Configuration	Observed Output
0	Baseline: Temperature = 0 With Grounding/Fall back	"Memory is the ability to remember past experiences, and a record of the learning process."
1	Ungrounded: Temperature = 1.0 Without Grounding/Fall back	"Memory is the ability to remember past experiences and serves as a record of the learning process. In the human brain, neuroplasticity lets us form new neural pathways, alter existing connections, and adapt to new information. For information to be retrievable, it must first be encoded into long-term memory."

As shown in Table III, the model produced a concise and direct response when using a temperature of 0 with grounding and fallback instructions. In contrast, the configuration with a temperature of 1.0 without grounding and fallback instructions generated a more detailed and elaborated response, including additional information about

memory. The results suggest that the change in configuration influenced the amount of detail and variation in the generated response. However, the additional information must be verified against the retrieved document context to determine whether it is supported by the knowledge base and can be considered accurate within the RAG system.

A. Out-of-Domain Queries

For the out-of-domain evaluation, the baseline configuration behaved as intended by returning the predefined fallback response, *"I cannot answer based on the provided domain data."* This indicates that the grounding and fallback instructions prevented the model from attempting to answer a question that was unsupported by the provided domain context. When the same out-of-domain query was tested at a temperature of 1.0 without the grounding and fallback rules, the model responded, *"I'm sorry, but I don't have enough information from the provided context to answer that question."* Although the predefined fallback instruction had been removed, the model still recognized that the retrieved context did not contain sufficient information and independently declined to provide an answer.

TABLE IV. OUT-OF- DOMAIN QUERY RESULTS

Temperate	Configuration	Observed Output
0	Baseline: Temperature = 0 With Grounding/Fall back	"I cannot answer based on the provided domain data."
1	Ungrounded: Temperature = 1.0 Without Grounding/Fall back	"I'm sorry, but I don't have enough information from the provided context to answer that question."

As shown in Table IV, the model declined to answer the out-of-domain query under both configurations. At a temperature of 0 with grounding and fallback instructions, the model returned the predefined fallback response, indicating that the requested information was not available in the provided domain data. At a temperature of 1.0 without grounding and fallback instructions, the model also recognized that the retrieved context did not contain sufficient information and generated its own refusal response. The results show that no hallucinated answer was observed in this test, even after removing the explicit grounding and fallback instructions. However, the baseline configuration demonstrated more controlled and predictable behavior because it followed the predefined fallback response exactly.

V. CONCLUSION

Based on my experience, prompt restrictions make the system more reliable because, instead of hallucinating or providing potentially inaccurate information, the system can acknowledge when it cannot answer a question based on the

available context like what happen to the out-of-domain experiment wherein the output presents that it cannot locate the answer of it doesn't enough information to provide answer.. As students and users, it is more helpful to know when the system does not have enough information to provide an accurate answer rather than receiving information that may be incorrect. This creates a safer and more trustworthy environment by reducing the possibility of presenting unsupported information as factual. The retrieval process further supports the reliability of the system because it retrieves information specifically from the documents and files stored in the knowledge base, which have been selected and verified for the intended domain, rather than relying solely on the model's general knowledge. Moreover, API-key protection contributes to the security of the system by preventing sensitive credentials from being directly exposed in the source code. Using environment variables and hidden input through getpass helps protect the API key from accidental exposure or unauthorized use. Overall, my experience with the RAG system showed that prompt restrictions, retrieval-based information, and basic API-key protection work together to create a chatbot that is more controlled, reliable, and secure for its users.

REFERENCES

- [1] OECD, *PISA 2022 Results (Volume I): The State of Learning and Equity in Education*. Paris, France: OECD Publishing, 2023, doi: 10.1787/53f23881-en.
- [2] M. Romero, C. Casadevante, and J. Santacreu, "Time Management, Fluid Intelligence and Academic Achievement," *Psychological Studies*, vol. 69, pp. 59–68, 2024, doi: 10.1007/s12646-023-00764-0.
- [3] "How am I going and where to next? Elaborated online feedback improves university students' self-regulated learning and performance," *The Internet and Higher Education*, vol. 55, Art. no. 100872, 2022, doi: 10.1016/j.iheduc.2022.100872.